

Freight Rate Prediction

Machine Learning Engineer assessment, Spotter

Fasih Ur Rehman | 18 September 2026 | Code: <https://github.com/Fasih-DS-AI/spotter-freight-rate-assessment> | Walkthrough: <https://www.loom.com/share/aef38079b00342b2af36746f4e08f382>

1. Summary

Task. Predict the posted rate of 12,000 truckload freight loads in November-December 2025 from lane, equipment, weight, date and a market index, after learning from 48,000 labelled loads in January-October 2025. Because every scoring load is later than every training load, this is a two-month-ahead forecast and it is validated as one.

Approach. Log rate-per-mile is decomposed into a small linear time component (trend, market index, quarter-end ramp, weekday) that can be extrapolated past the training window, plus a LightGBM structural component fitted on the residual with load features only. The quote signal provided in the data is deliberately excluded: it is an exact copy of the rate in some months, degraded in others, and pure noise in the scoring period.

Result. On a chronological holdout (train January-August, test September-October) the model reaches a mean absolute percentage error of 1.47% on genuine loads (MAE \$34), against 3.65% for an equipment-by-distance-band median and 9.87% for quote signal times distance. A five-window rolling backtest gives 1.31% to 1.60% with absolute bias under 1%. About 1.4% of targets are corrupted by large multiplicative errors and are reported separately, since no model can predict them.

Deliverables. validation_predictions.csv (12,000 rows, validated by the provided scorer), the fixed December chart produced by score.py (section 6), and a repository that reproduces everything with one command (appendix).

2. The data and what it revealed

Both files share the same schema: pickup and delivery city with coordinates, distance, equipment (Dry Van, Reefer, Flatbed), weight, date, a market index and a quote signal. The development file has the posted rate; the scoring file does not. The coordinates are synthetic but internally consistent per city, and distances are consistent with them, so both are used as given.

file	loads	dates	cities	posted_rate
train_test.csv	48,000	1 Jan - 31 Oct 2025 (304 days)	64	yes
validation.csv	12,000	1 Nov - 31 Dec 2025 (61 days)	72 (8 unseen in training)	no

Table 1. The two files. Every scoring date is later than every training date.

2.1 The posted rate is rate-per-mile times distance

Rate-per-mile falls with distance (elasticity about -0.13 in log-log terms), is about 8% higher for Flatbed and 12% higher for Reefer than for Dry Van, rises about 12% from the lightest to the heaviest loads, and is a few percent higher for lanes touching the South-Central states than the West Coast. Modelling rate-per-mile rather than the dollar rate removes most of the scale and makes the remaining structure nearly additive in log space, which is what the model exploits.

2.2 The quote signal is a trap

In January-March, June and September, `quote_signal` times distance reproduces `posted_rate` within 1% for 98% of loads (Figure 1). In April-May, July-August and October it is a degraded signal: lower on average, with the sign of its relationship to distance flipped (Figure 2, middle). In the scoring file it has no relationship with distance, equipment, weight or region at all: a regression of `quote_signal` on those features explains 0.2% of its variance, versus 90% in the exact months (Figure 2, right). A model trained on the development data learns to lean on it (it is the single best in-sample feature) and then fails on exactly the loads that are scored. Section 4 quantifies the damage; the feature is excluded from the model.

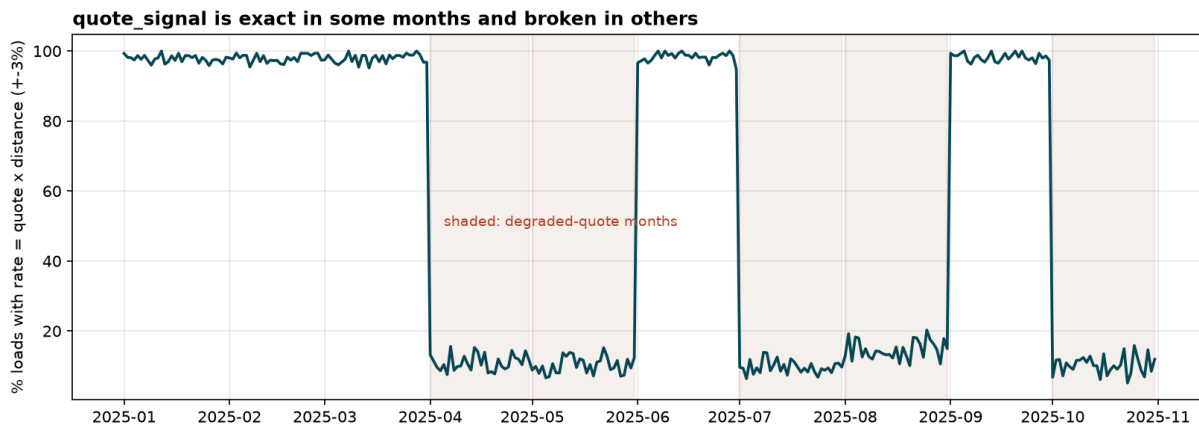


Figure 1. Share of loads per day whose posted rate equals `quote_signal` x distance within 3%. The signal switches between an exact regime and a broken one at month boundaries.

In the scoring period `quote_signal` carries no structure at all

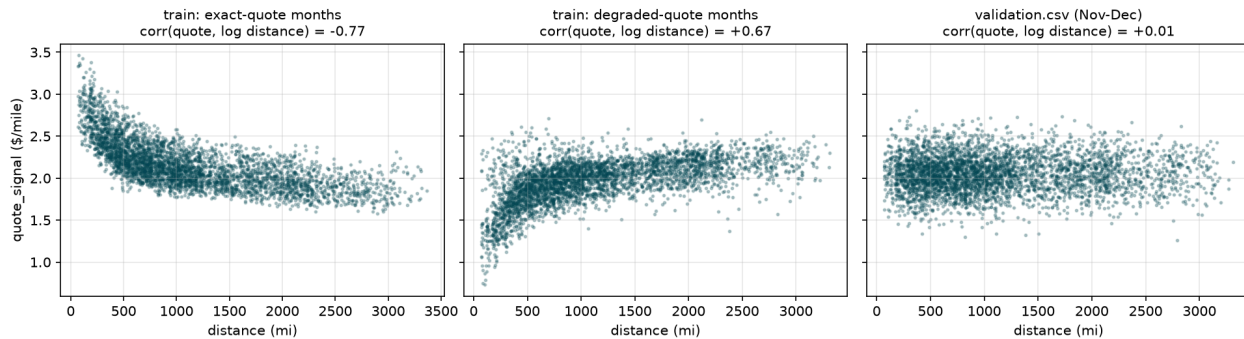


Figure 2. `quote_signal` against distance in the exact months, the degraded months and the scoring file. In the scoring file it is noise around \$2.05 per mile.

2.3 About 1.4% of the training targets are corrupted

675 development loads (1.41%) carry a posted rate that has been multiplied by a factor between about 2.3 and 5.6, or divided by one. Because the quote signal is exact in half the months, the corruption can be characterised precisely there: genuine loads have 1.57 to 3.57 dollars per mile, corrupted ones fall below 1.25 or above 4.2, and there is an empty gap between the populations (Figure 3). A rate-per-mile filter of 1.2 to 4.0 therefore removes them cleanly, and the same rule identifies the corrupted loads in the months where the quote cannot be used. The scoring file presumably contains the same 1.4%, which sets a floor on any raw error metric; all metrics in this report are given on all loads and on genuine loads.

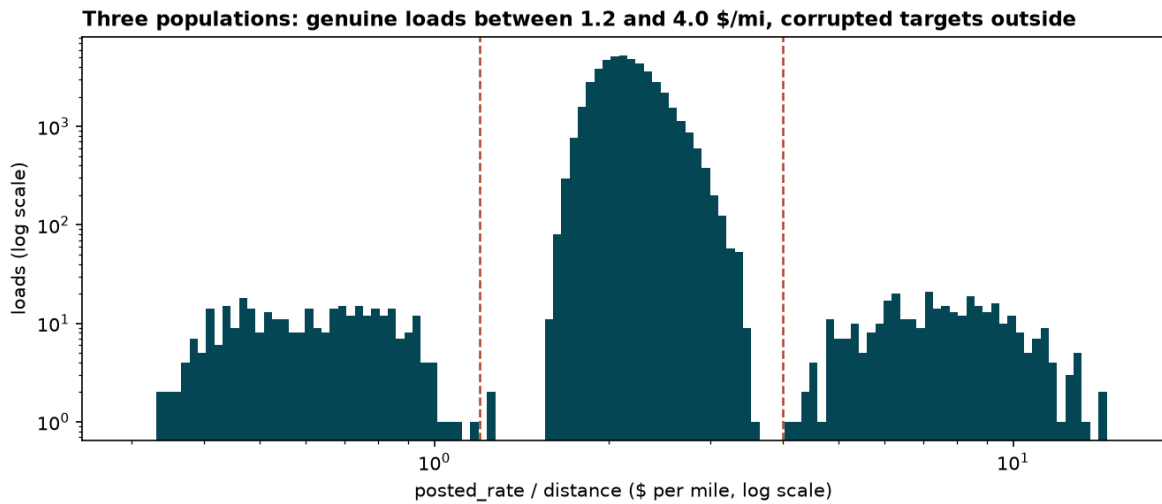


Figure 3. Distribution of `posted_rate / distance` on log axes. The dashed lines are the filter bounds; everything outside them is a corrupted target.

2.4 The market index is a daily market series with a weekly cycle

`market_index` varies mostly between days (standard deviation 0.165 across daily means) and very little within a day (0.025), so it is a market-wide series with a small per-load jitter. It has a strong weekly cycle (peak Thursday, trough Sunday, about 19% peak-to-trough) on top of a slow swing that peaked in May and settled around 0.93 in November-December, lower than the development-period mean of 1.08 (Figure 4). Rates respond to it with an elasticity of about 0.15 log-points per index unit. The daily series is used to impute the 0.8% of loads with a missing value and to supply values for the fixed December chart.

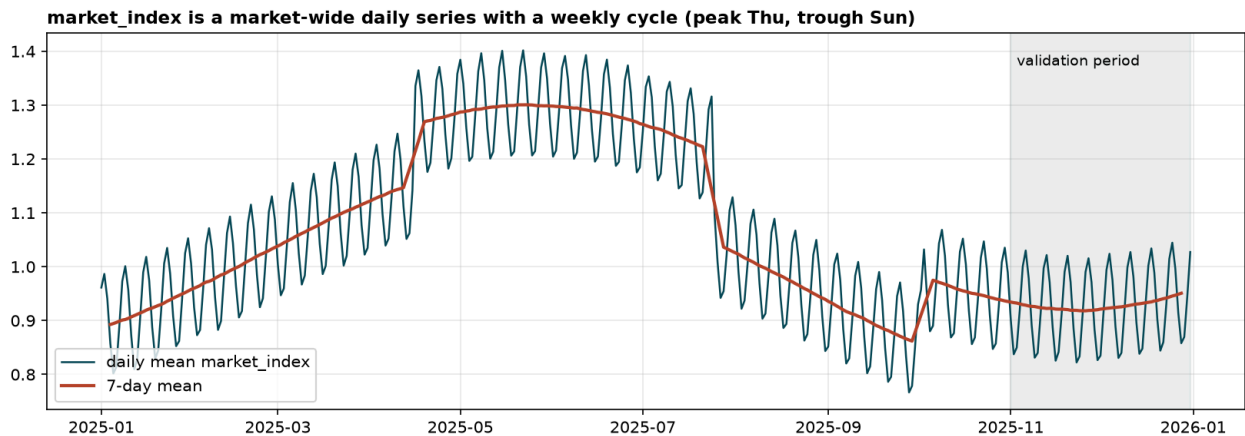


Figure 4. Daily mean market index with a 7-day mean; the shaded area is the scoring period.

2.5 Rates ramp up through quarter-end months, on top of a slow trend

After controlling for load features and the market index, rates rise steadily through March, June and September, ending the month about 4% above where they started, while other months are flat (Figure 5). There is also a slow upward drift of about 0.6% per month over the year. Both effects matter for the scoring period: December is a quarter-end month, and the trend has to be extended two months past the data.

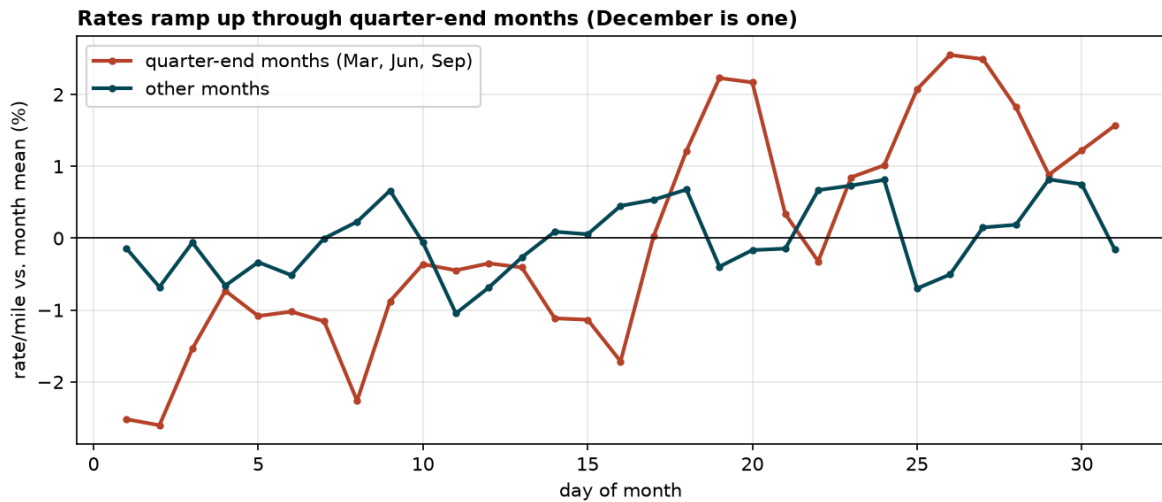


Figure 5. Rate-per-mile relative to its month mean by day of month, quarter-end months versus the rest (model-free view; the fitted ramp is +4.1%).

3. Data-quality issues and how they were handled

Every repair leaves an indicator column and is counted in a quality report generated by the pipeline (reports/data_quality.md). Cleaning is applied identically to both files, except that the corrupted-target rule needs the target and so applies to the development file only.

issue	count	diagnosis	handling
Negative weight	292 train / 145 validation	Sign flip: the magnitudes match the normal weight distribution	$\text{abs}(\text{weight}) + \text{indicator}$
Missing weight	300 / 165	Missing at random	Left as NaN for LightGBM (native handling) + indicator; median-filled in the linear stage
Missing market_index	374 / 249	Market-wide daily series	Same-day mean of the other loads (day-of-week profile fallback for unseen dates)
Corrupted posted_rate	675 (1.41%) / unknown	Multiplicative x2.3-5.6 up or down	Flagged by rate-per-mile outside 1.2-4.0; excluded from fitting, kept in evaluation
quote_signal	all rows	Exact in 5 months, degraded in 5, noise in scoring file	Excluded from the feature set
Cities only in validation	8 cities, 1,447 loads (12%)	Allentown, Charlotte, Chicago, Jackson, Knoxville, Laredo, Norfolk, San Diego	No city identity features; geography via coordinates and lane geometry (tested in 4.4)
Clipped coordinates, distance floor	Boston/Providence lon -69.5; 69 loads at 70 mi	Synthetic generator limits	Distances are consistent with the coordinates, so used as given

Table 2. Data-quality issues, their diagnosis and the fix applied.

4. Validation design and data split

The scoring loads are all later than the development loads, so a random split would be misleading in two ways: it would let the model see the same days it is tested on, and it would reward the quote signal. Every split used here is chronological and every test window is later than everything the model was trained on.

4.1 Primary holdout

Train on 1 January to 31 August (38,477 genuine loads after excluding corrupted targets) and test on 1 September to 31 October (9,523 loads, corrupted targets included so that the reported numbers are honest). This is exactly the horizon of the real task: two months of predictions starting the day after the training data ends. Three reference predictors are evaluated on the same split: the global median rate-per-mile times distance, the median rate-per-mile within equipment and distance band, and the quote signal times distance.

model	MAE (all)	MAPE (all)	R ² (all)	MAE (genuine)	MAPE (genuine)	bias (genuine)
hybrid (this submission)	\$90	3.99%	0.830	\$34	1.47%	-0.73%
median rate/mile x distance	\$257	11.63%	0.799	\$204	9.27%	-1.46%
equipment x distance-bin median rate/mile	\$135	6.16%	0.825	\$79	3.65%	-0.70%
quote_signal x distance (the tempting leak)	\$246	12.11%	0.782	\$192	9.87%	-4.80%

Table 3. Holdout results, September-October 2025. 'Genuine' excludes the 1.5% of corrupted targets in the test window; bias is the mean signed percentage error.

The quote-signal predictor is the cautionary case: it explains posted rates almost perfectly in the exact months, yet on the holdout it is worse than a distance-band median because October is a degraded-quote month. In the scoring period, where the signal is pure noise, it would be worse still. By month, the model's genuine-load MAPE is 1.32% in September and 1.63% in October, with a bias of -1.17% in October: the second month ahead is harder, as expected for a forecast, and the model slightly under-predicts it because the trend is a little steeper late in the year than the linear fit.

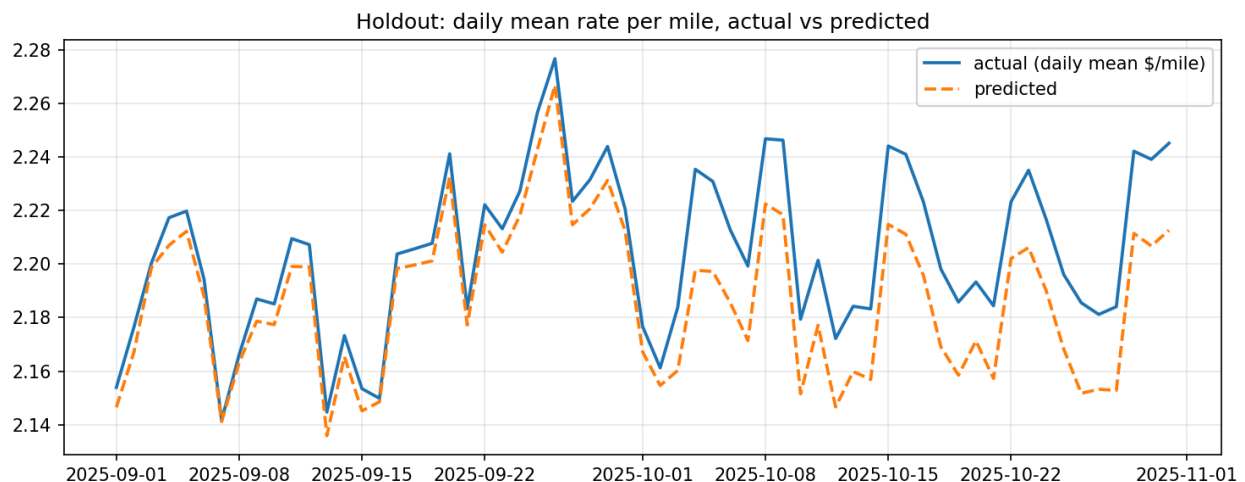


Figure 6. Daily mean rate-per-mile on the holdout, actual versus predicted. The model tracks the weekly market cycle and the September quarter-end ramp.

4.2 Rolling-origin backtest

One holdout is one draw. To check that the result is stable across the quote regimes and the market cycle, the same procedure is repeated over five two-month windows, each trained on everything strictly before it (Table 4, Figure 7). The fitted trend and quarter-end ramp are reported per window to show the time component is stable as data accumulates.

test window	train loads	MAE (all)	MAE (genuine)	MAPE (genuine)	bias	trend / month	quarter-end ramp
May-Jun 2025	19,110	\$90	\$36	1.48%	-0.64%	+0.33%	+3.9%
Jun-Jul 2025	24,023	\$92	\$34	1.38%	-0.30%	+0.53%	+3.8%
Jul-Aug 2025	28,806	\$90	\$37	1.60%	+0.95%	+0.72%	+3.9%
Aug-Sep 2025	33,718	\$81	\$30	1.31%	+0.14%	+0.55%	+4.1%
Sep-Oct 2025	38,477	\$90	\$34	1.47%	-0.73%	+0.52%	+4.0%

Table 4. Rolling-origin backtest. Mean MAE \$89, mean genuine-load MAPE 1.45%, mean absolute bias 0.55%.

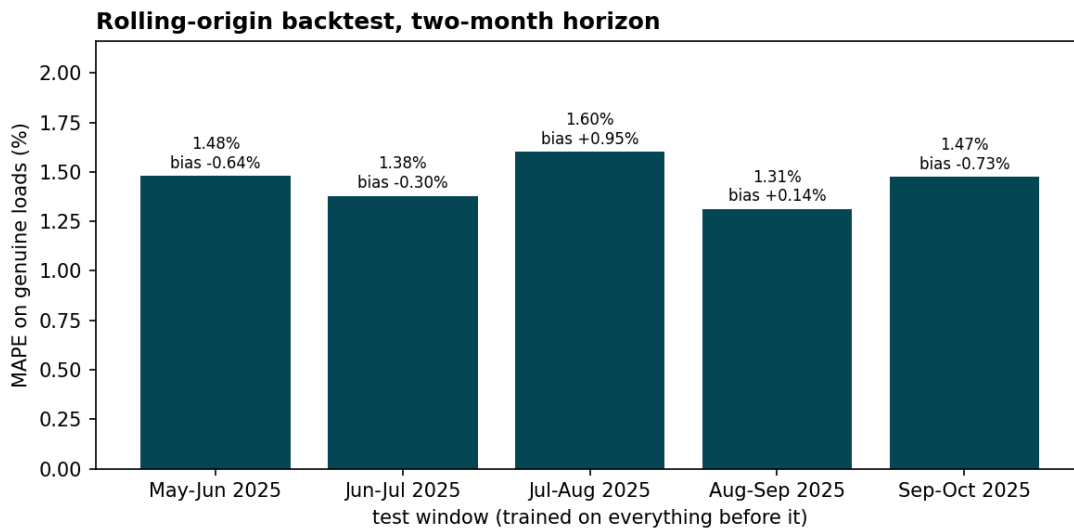


Figure 7. Genuine-load MAPE and bias per backtest window.

4.3 Why the time component is separated from the trees

A single LightGBM given the same features plus the date features is the natural alternative. On the same five windows it reaches a genuine-load MAPE of 1.87% with a mean absolute bias of 1.24%, against 1.45% and 0.55% for the hybrid. Two things go wrong with the single model: trees cannot extrapolate the trend, so every future window is predicted at the last level seen; and the daily market index acts as a day identifier that lets trees memorise daily levels in-sample, which is why weekday and day-of-month features receive no importance in that model and the ramp is lost out of sample. Separating a linear time component that is fitted with structural controls, and giving the trees no date at all, fixes both.

4.4 Unseen cities

Twelve percent of scoring loads touch a city that never appears in training. To measure the cost, eight training cities (Tulsa, Washington, San Antonio, Memphis, Toledo, Hartford, Bakersfield, New York) were removed from the training data entirely and the holdout was scored separately for loads touching them: genuine-load MAPE

1.68% on those loads against 1.51% on the rest, with the same bias. Coordinates and lane geometry carry the regional effect; a city-identity feature was tried and does not help even for seen cities.

4.5 Leakage controls

- No feature is derived from the target other than the corrupted-target flag, which is used only to choose fitting rows in the training window.
- The market calendar is built from the market_index feature, which is available in both files; no target information enters it.
- No target encoding of cities or lanes (tested; no gain, and unsafe for unseen cities).
- Hyper-parameters were chosen on the holdout only; the backtest was run once with the final configuration.

5. Model

Decomposition. $\log(\text{rate} / \text{distance}) = \text{time}(\text{date}, \text{market index}) + \text{structure}(\text{load})$. Multiplying the exponentiated sum by distance gives the rate.

Time component. Ordinary least squares on log rate-per-mile with a linear trend, the load's market index, a quarter-end ramp (zero outside March/June/September/December, rising from 0 to 1 through those months), a day-of-month term and weekday indicators, plus coarse structural controls (log distance and its square, weight, equipment) so that the time coefficients are not confounded. Only the time terms are kept. Fitted on all development data: trend +0.59% per month, market elasticity 0.147 log-points per index unit, quarter-end ramp +4.1% first to last day, weekday effects within -0.62% to -0.02% of Monday. The trend is extended linearly past October; damped and frozen alternatives are available as switches and were worse on the backtest, as was a quadratic trend (which over-predicts historically).

Structural component. LightGBM regression on the residual with distance, log distance, equipment, weight and a missing-weight indicator, pickup and delivery coordinates, their differences and days to quarter end. 63 leaves, minimum 40 loads per leaf, learning rate 0.03, 1,500 rounds, 80% feature and row subsampling, L2 regularisation 1.0; three boosters with different seeds are averaged. No date, trend or market feature is visible to the trees.

feature	gain share
distance	50.2%
equipment	21.0%
log_distance	16.6%
weight	4.8%
delta_lon	1.7%
pickup_lat	1.5%

Table 5. LightGBM gain share of the structural features (final model). Remaining features (delivery_lat, pickup_lon, delivery_lon, delta_lat, days_to_quarter_end, weight_missing) share the other 4.3%.

Alternatives tried and rejected. Using the quote signal (fails out of time); predicting the dollar rate directly instead of rate-per-mile (worse scale handling); Huber and L1 objectives (no gain once corrupted targets are excluded); city as a categorical feature and smoothed target encoding of cities and lanes (no gain, unsafe for unseen cities); LightGBM linear trees for extrapolation (worse); a smoothed market index in place of the load's own value (slightly worse); a quadratic trend (over-predicts on the backtest).

6. Fixed December prediction chart

The chart required by the brief holds every input fixed (Lexington to Fort Wayne, 360 miles, Dry Van, 32,000 lb) and varies only the date across December 2025. The inputs were expanded to the model's schema exactly as a real load would be: coordinates from the city table, the market index from the daily calendar (validation.csv contains loads on every December day, so no market value is invented), quote signal left empty because the model does not use it. The predictions were written into the provided template and the chart was produced by the unmodified score.py.

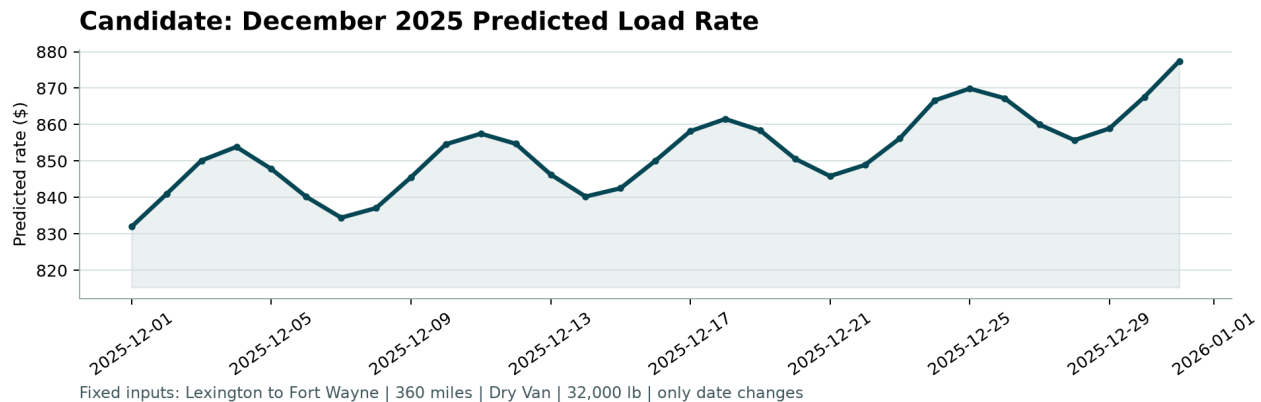


Figure 8. Output of score.py: predicted rate for the fixed load on each day of December 2025.

Reading the shape: the level starts at \$832 on 1 December and ends at \$877 on 31 December (\$832 to \$877 overall). The rise is the quarter-end ramp (December is treated like March, June and September) plus two months of the fitted trend; the waves are the weekly market cycle, peaking mid-week when the market index peaks. For scale, the development data contains 21 Dry Van loads on this lane at \$758 to \$934 (median \$808), most of the later ones between \$800 and \$900, so the December level is consistent with the lane's history. No holiday effect is modelled because the development data shows none around New Year, Memorial Day, Independence Day or Labor Day; if Christmas week behaves differently in reality, the chart will not show it.

7. Final predictions

The final model is refitted on all development data (47,325 genuine loads; 675 corrupted targets excluded) and applied to validation.csv after the same cleaning. The template's row order is preserved and every prediction is positive. Predicted rates have a mean of \$2,395, a median of \$2,073 and range from \$188 to \$7,138, in line with the development-data distribution. The provided scorer validates the file (12,000 ids, two columns) and the December file (31 days, fixed inputs).

8. Limitations and next steps

- The trend is extrapolated linearly for two months. The backtest supports this, but the late-year slope looks slightly steeper than the full-year fit; if the scoring metric shows a negative bias, a locally estimated slope is the first thing to try.
- December is assumed to behave like the other quarter-end months. That is the natural reading of the data, but it is an assumption stated here rather than something the data can prove.
- Corrupted targets cannot be predicted, and they are presumably present in the scoring file at the same 1.4% rate; roughly half of any raw MAE on this data comes from them.

- Holiday effects are absent from the development data and therefore from the model.
- Prediction intervals were not produced; quantile boosters on the same residual would add them at little cost.

Appendix: reproducing the results

```
python -m pip install -r requirements.txt
python -m pip install -e .
python -m freight_rate.cli all          # quality, eda, validate, train, predict, score
python -m pytest -q                    # 12 tests on synthetic data
python scripts/extra_experiments.py    # section 4.3 and 4.4
python scripts/build_report.py         # this document
```

Repository layout: `src/freight_rate/` (config, data, features, model, validate, eda, predict, cli), `tests/`, `reports/` (tables and figures used here), `outputs/` (predictions), `data/` (provided files), `docs/` (the brief), `score.py` (unmodified).